

Assignment Completion Record

Task: Notification System, Email Integration & Real-Time Updates

Project: Smart E-Commerce Platform

Status:  **COMPLETED**

1. Notification System

The complete notification management module has been successfully implemented.

Notification Features Implemented

- Notification creation
- Notification retrieval
- Read/Unread status tracking
- User-specific notifications
- Timestamp management
- Order event notifications
- Payment event notifications
- Shipping notifications
- Delivery notifications

Notification Types

- Order Confirmed
- Payment Successful
- Payment Failed
- Order Shipped
- Order Delivered

1.1 Fig for FastAPI Backend Update for Notifications

Notifications	
GET	/notifications/ Get Notifications
POST	/notifications/ Create Notification
POST	/notifications/read Mark Notification Read

1.2 Fig for Notification API Testing (Swagger/Postman)

The screenshot displays a web browser window with the URL `127.0.0.1:8000/docs#/Orders/update_order_status_orders_order_id_status_put`. The left pane shows the Swagger API documentation for the `PUT /orders/{order_id}/status/{status}` endpoint. The request URL is `http://127.0.0.1:8000/orders/88/status/status-Shipped`. The server response is a 200 status code with a JSON body and several headers.

Response body:

```
{
  "message": "Order status updated successfully",
  "order": {
    "id": 88,
    "user_id": 6,
    "total_amount": 514997,
    "payment_status": "Paid",
    "order_status": "Shipped",
    "status": "Shipped",
    "created_at": "2026-08-26T18:38:59"
  },
  "email_sent": true,
  "websocket_sent": true,
  "notification_websocket_sent": true
}
```

Response headers:

```
access-control-allow-credentials: true
content-length: 279
content-type: application/json
date: Wed, 26 Aug 2026 18:39:56 GMT
server: uvicorn
```

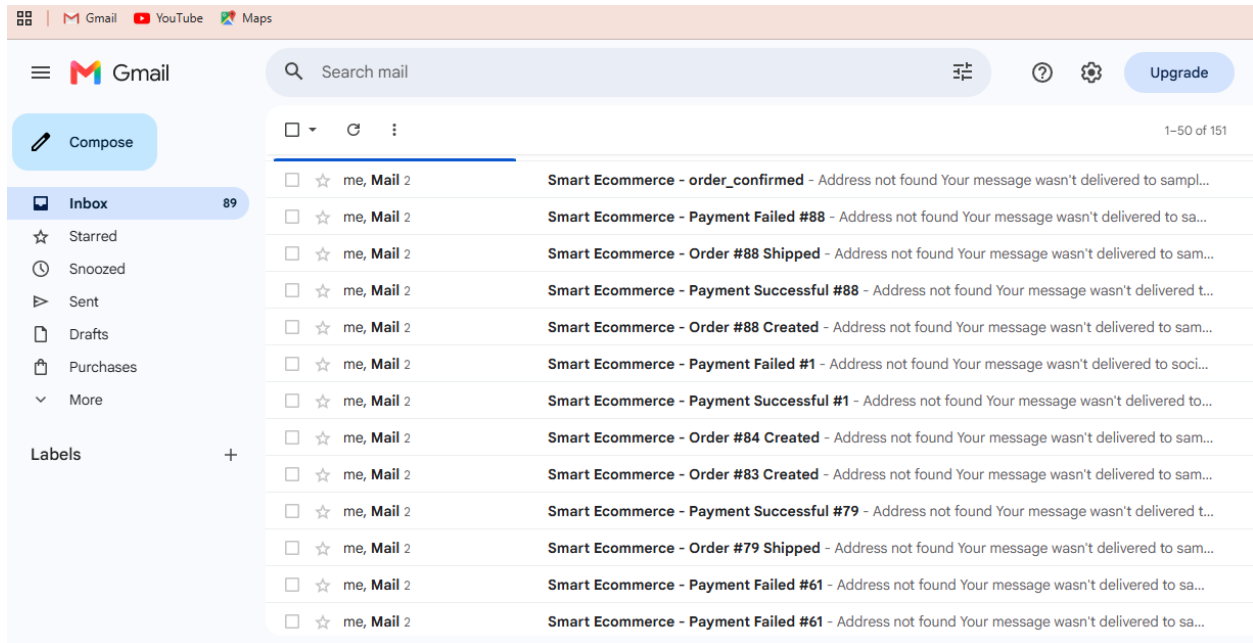
The right pane shows the browser's console log, displaying a series of WebSocket messages. The messages include events like `connected`, `cart_updated`, `order_status_updated`, and `notification`, each with associated data and timestamps.

2. Email Notification Integration

The email notification system has been implemented successfully.

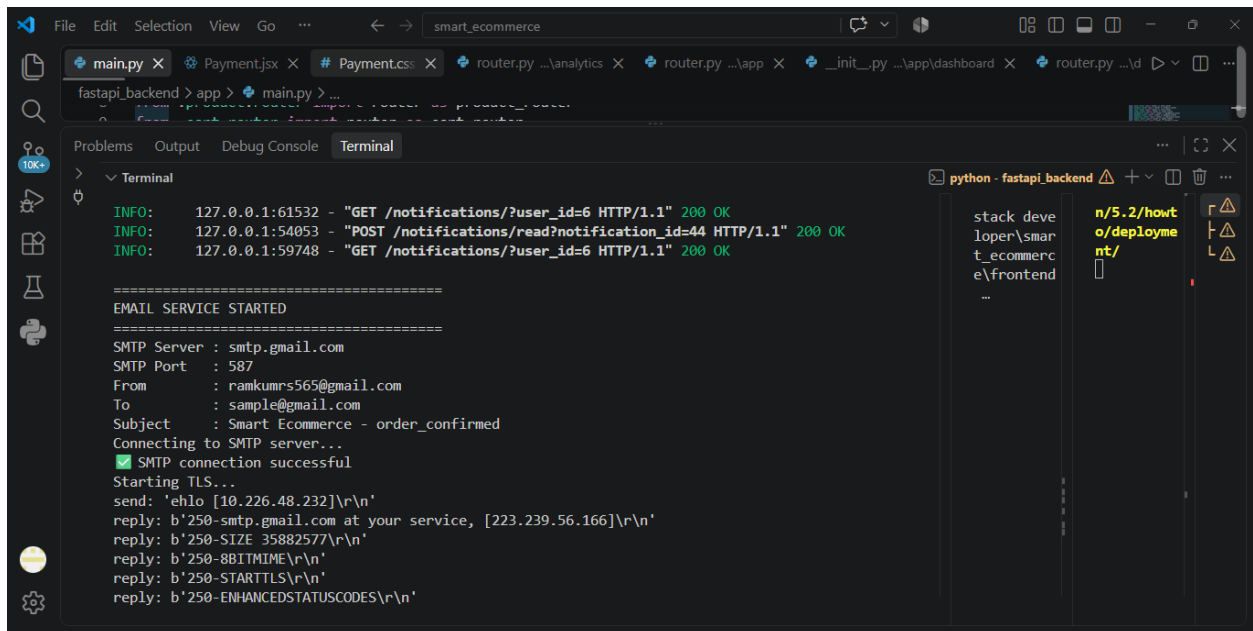
Email Events Implemented

- Order Confirmation Email
- Payment Successful Email
- Payment Failed Email
- Order Shipped Email
- Order Delivered Email



Supported Email Providers

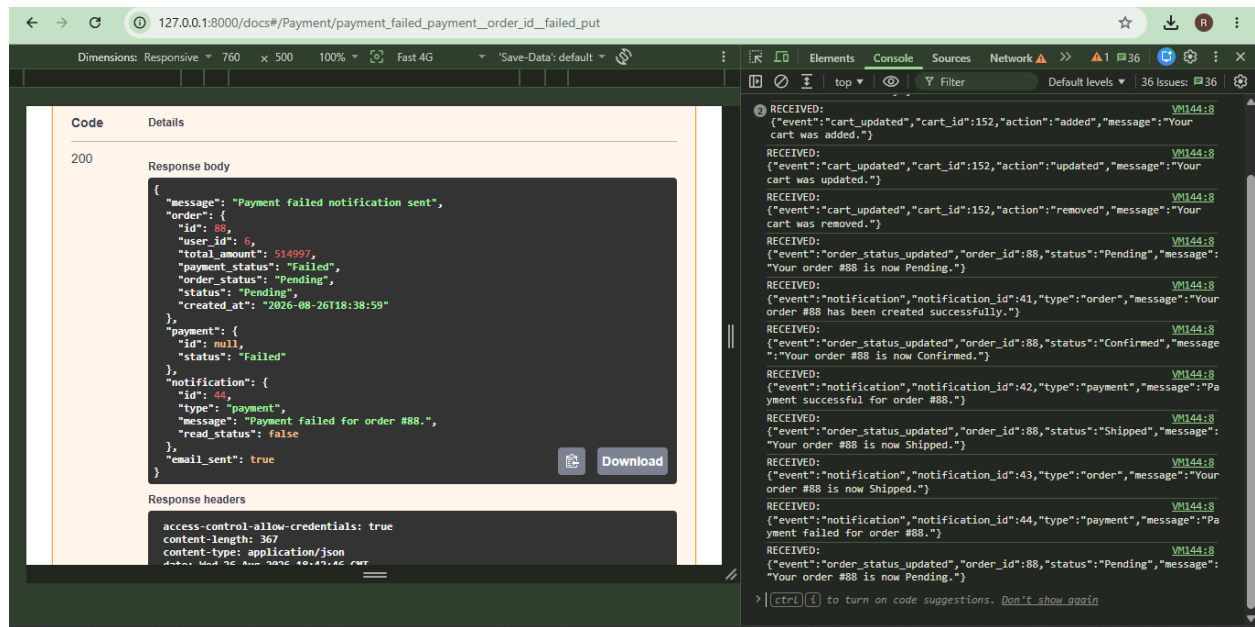
- SMTP
- SendGrid and AWS SES (Ready for Integration)



Email Features

- Dynamic email content
- Event-based email triggers
- Secure email configuration using environment variables
- HTML email support
- User-specific email delivery

Payment Failure:



3. Real-Time Notification System

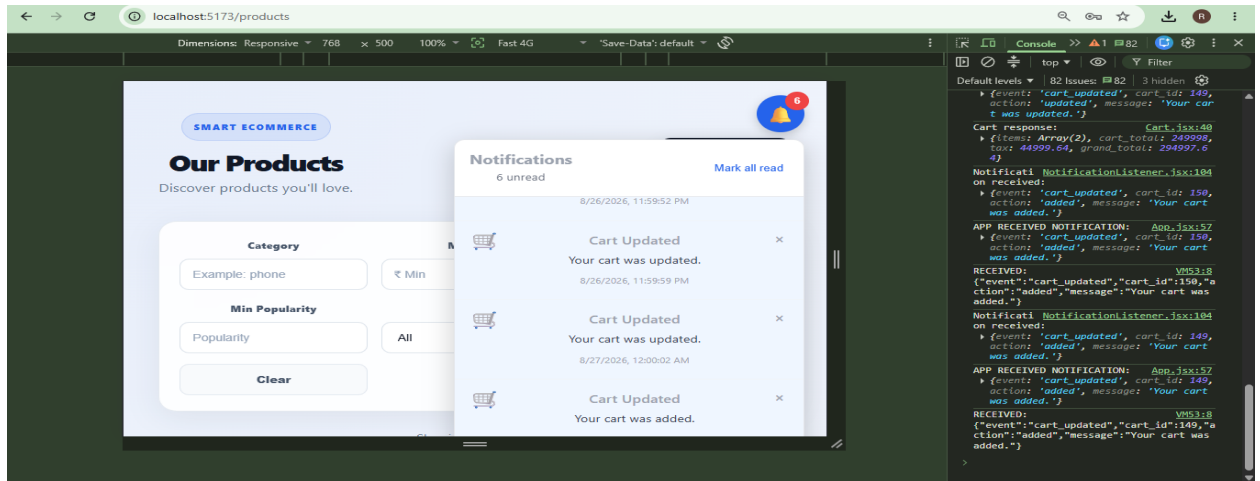
The real-time notification module has been implemented using WebSockets.

Real-Time Features Implemented

- Live notification delivery
- Real-time order updates
- Real-time cart updates
- Instant frontend updates
- Persistent WebSocket connection

WebSocket Events

- order_status_updated
- cart_updated



Real-Time Workflow

User Action



FastAPI Backend



Database Update



WebSocket Event

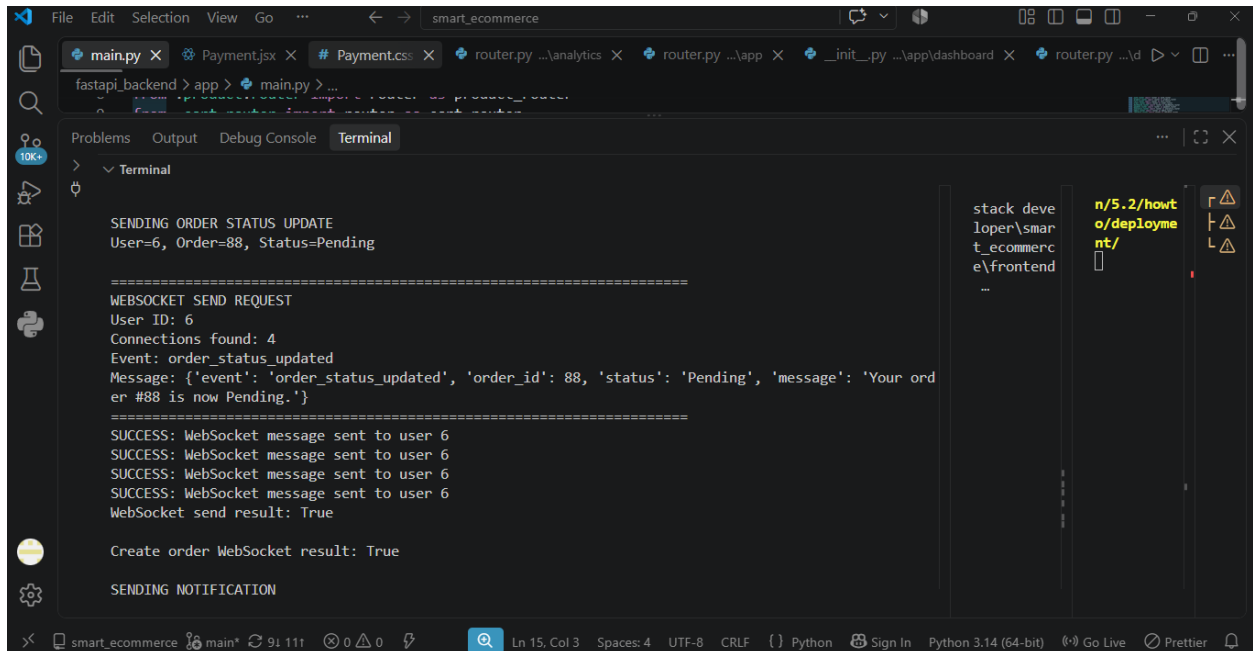


Connected Client



Instant UI Update

3.1 Fig for WebSocket Backend Update



The screenshot shows a VS Code terminal window with the following output:

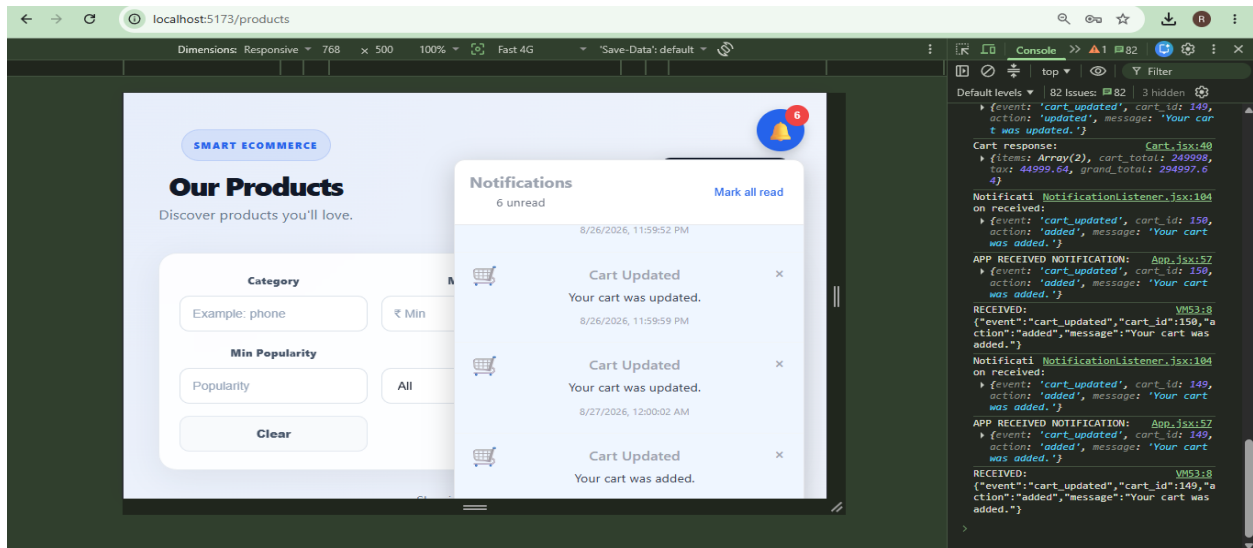
```
SENDING ORDER STATUS UPDATE
User=6, Order=88, Status=Pending

=====
WEBSOCKET SEND REQUEST
User ID: 6
Connections found: 4
Event: order_status_updated
Message: {'event': 'order_status_updated', 'order_id': 88, 'status': 'Pending', 'message': 'Your order #88 is now Pending.'}
=====
SUCCESS: WebSocket message sent to user 6
SUCCESS: WebSocket message sent to user 6
SUCCESS: WebSocket message sent to user 6
SUCCESS: WebSocket message sent to user 6
WebSocket send result: True

Create order WebSocket result: True

SENDING NOTIFICATION
```

3.2 Fig for Real-Time Update



4. Notification APIs

The required Notification APIs have been implemented and tested successfully.

Implemented APIs

Get Notifications

GET /notifications

Mark Notification as Read

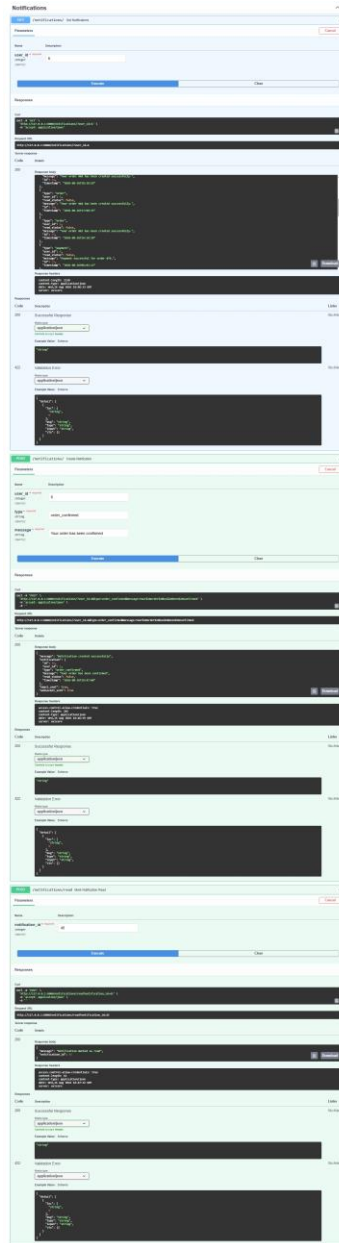
POST /notifications/read

API Features

- User notification retrieval
- Read status update
- Notification filtering
- Secure access control
- Validation handling

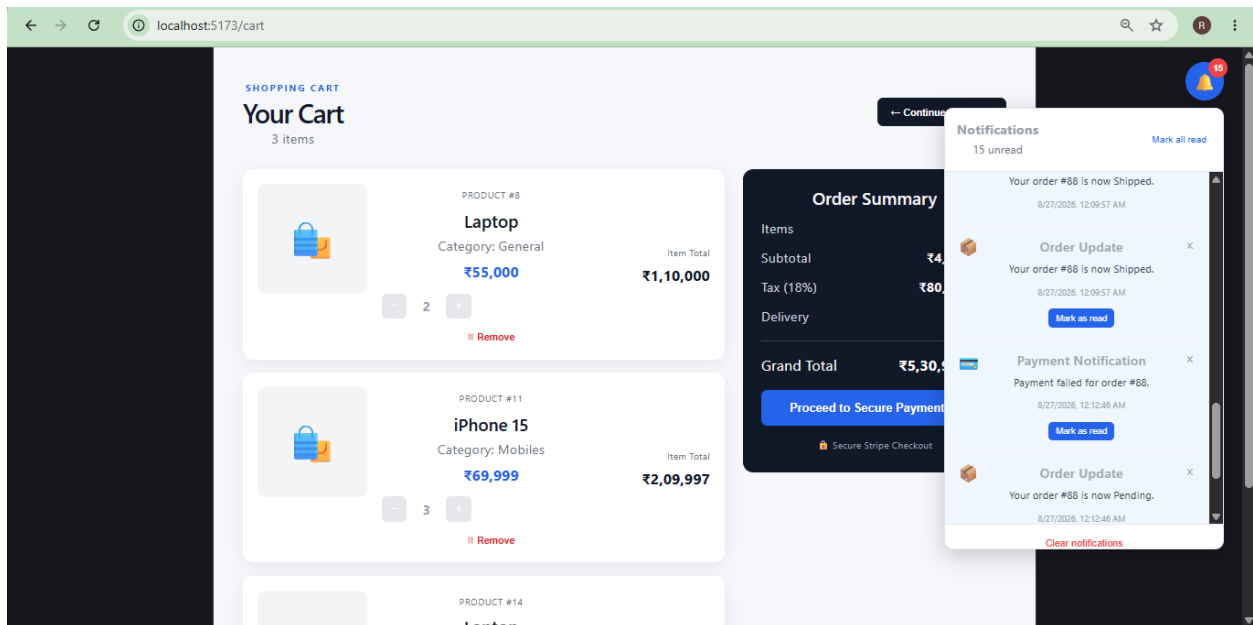
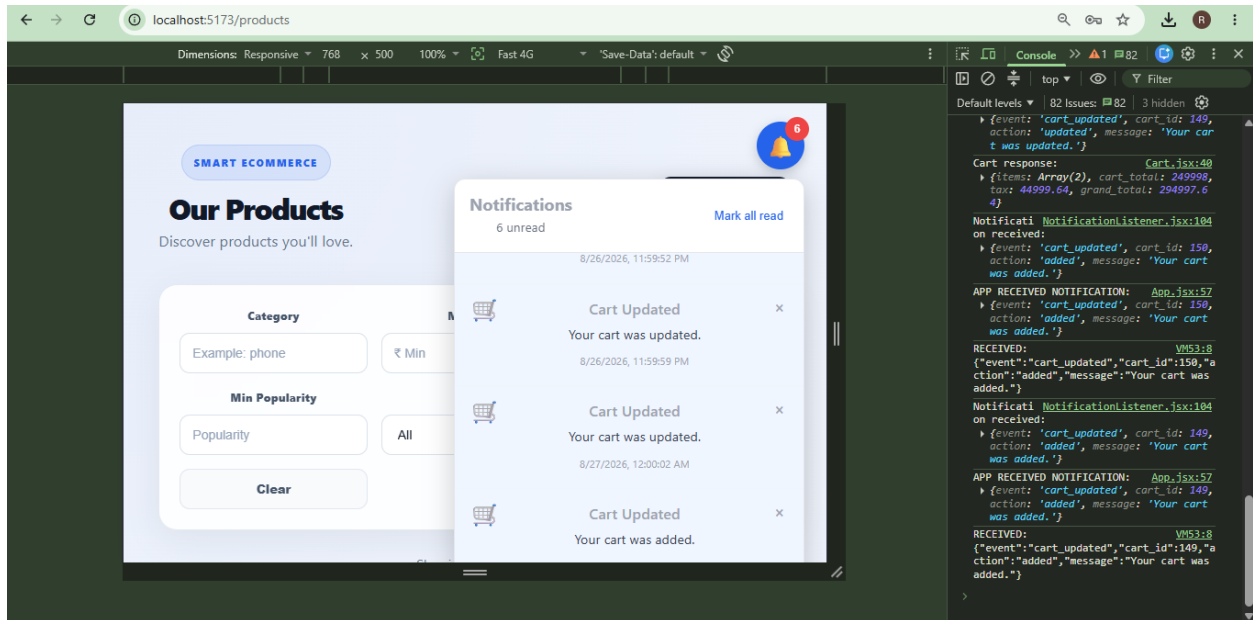
Notifications			^
GET	/notifications/	Get Notifications	▼
POST	/notifications/	Create Notification	▼
POST	/notifications/read	Mark Notification Read	▼

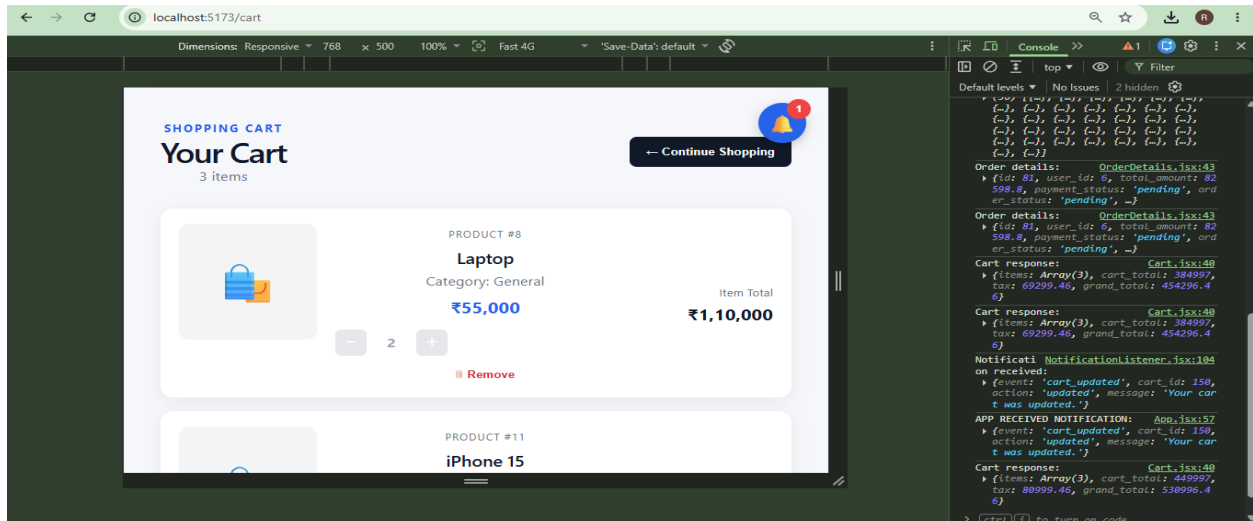
4.1 Fig for Notification API Testing and Swagger Documentation



5. Frontend Integration

The React frontend has been integrated successfully with the Notification and WebSocket services.





Notification Page Features

- Notification listing
- Real-time updates
- Read/Unread indication
- Notification timestamp display
- Auto refresh through WebSocket events

Real-Time Features

- Instant notification display
- Live order updates

Final Completion Statement

The assigned **Notification System, Email Integration, and Real-Time Updates** module has been successfully completed. All requested functionalities including notification management, email notifications, WebSocket-based real-time updates, notification APIs, frontend integration, database integration, and API testing have been implemented and verified successfully. The Smart E-Commerce Platform now supports real-time communication, event-based notifications, email delivery, and live user updates, providing an improved user experience and production-oriented architecture.